

Project Final Report: Multi-Platform Educational Infrastructure (LearningApp)

Course/Project Context: Scalable Educational Platform Infrastructure

Development Team: Team 3

Date: June 5, 2026

Team Members & Responsibilities

- **Ecem Nur Özen (Technical Lead & Fullstack Engineer):** Responsible for the complete technical development of the project ecosystem. This includes backend API architecture, database schema design, JWT authentication system, Flutter mobile application development, React admin dashboard development, system integration, software engineering processes, and overall infrastructure implementation.
- **Karya Kinyas (Content Strategist & Organizer):** Responsible for creating and organizing educational content within the platform, including lesson structures, topic flows, and quiz materials through the admin dashboard.
- **Betül Doğan (Learning Resource Manager):** Responsible for preparing and managing learning materials, educational resources, and platform content organization using the admin management system.
- **Gizem Gürtürk (Instructional Designer):** Responsible for educational content contribution, lesson planning, quiz preparation, and maintaining instructional materials through the admin panel.

Executive Summary

This report documents the architectural design, technical implementation, and deliverables of the **LearningApp** ecosystem. The system features a modular architecture comprised of an ASP.NET Core Web API backend, a PostgreSQL relational database, a cross-platform Flutter mobile client, and a React-based administration dashboard.

Key achievements include the implementation of secure JWT-based authentication, modular content delivery, real-time user progress tracking, an achievements engine, and a dynamic AI-powered question tutor powered by the Gemini API.

1. Introduction

Modern educational technologies require scalable, highly responsive, and decoupling infrastructures to serve diverse client applications (e.g., student mobile apps, admin dashboards, web clients) while keeping business logic and security policies centralized. Monolithic systems suffer from high maintenance overhead, tight coupling, and deployment limitations.

To address these limitations, this project implements a fullstack educational ecosystem using a **decoupled, modular architecture**. All communication between the administrative frontend, the mobile client, and the database passes through a centralized REST API.

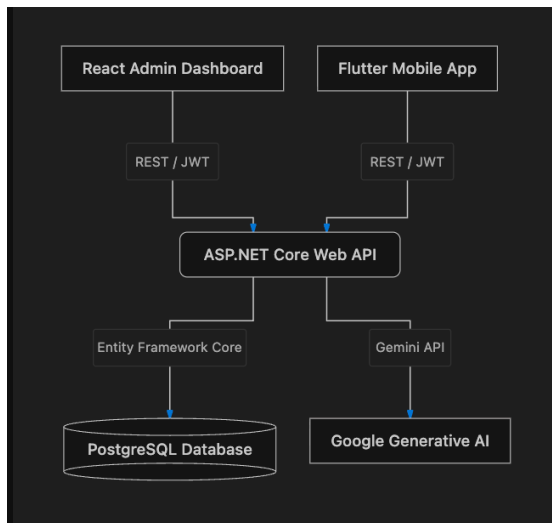
2. Problem Definition

Traditional fullstack architectures suffer from several core software engineering pain points:

- **Tight Coupling:** Frontends directly interfacing with databases or sharing heavy logic states, causing single-point failure risks.
- **Lack of Administrative Tools:** Creators having to seed content manually via database SQL scripts instead of secure dashboards.
- **Scalability Obstacles:** Inability to scale client-facing resources separate from admin modules or server processes.
- **Weak Authentication and Security:** Lack of role-based authorization across multiple clients, risking content leakage (e.g., exposing answer keys to clients).

3. Proposed Solution

The proposed system separates concerns into three distinct, self-contained layers:



1. **Centralized Backend (ASP.NET Core Web API):** Serves as the single source of truth, enforcing business logic, database migrations, authentication, achievements, and third-party AI connections.
2. **Mobile Client (Flutter):** Provides students with an immersive, gamified, and responsive learning interface (learning logs, progress tracks, active quizzes, and AI tutor explanations).
3. **Administration Dashboard (React):** Provides creators and editors with custom CRUD screens to organize topic structures, write markdown chapters, and design quiz options

4. System Architecture & Technologies

4.1 Backend Infrastructure

- **Framework:** ASP.NET Core (.NET 10.0) Web API.
- **Data Access:** Entity Framework Core (EF Core) utilizing Code-First migrations.
- **Database:** PostgreSQL relational database.

- **Security:** Role-based JWT (JSON Web Tokens) Authorization filters ([Authorize(Roles = "Admin, Student")]).
- **External Integrations:** Google Generative AI (Gemini 1.5 Flash API) for generating real-time motivational and contextual explanations for quiz errors.

4.2 Mobile Client

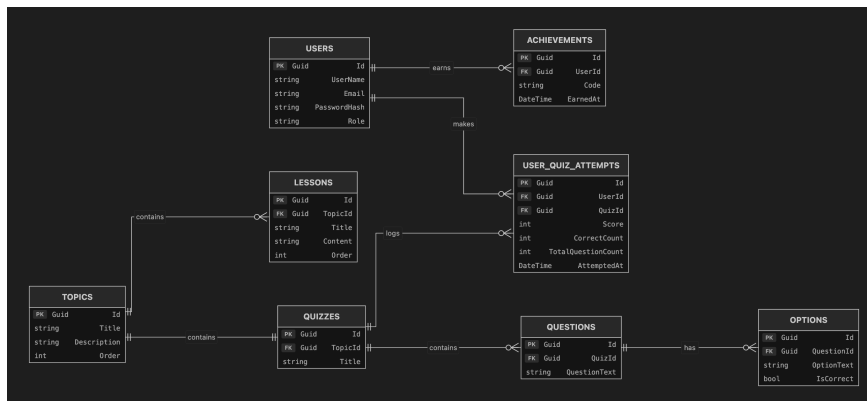
- **Framework:** Flutter (Dart).
- **State Management:** Provider pattern managing unified AppState.
- **Networking:** HTTP Client using REST principles, handling active tokens and request headers dynamically.
- **UI/UX Concept:** Fairytale and gamified aesthetics (glowing containers, achievement modals, custom badges, and smooth progress tracking).

4.3 Administrative Panel

- **Framework:** React.js.
- **State Management:** Context API & custom hooks.
- **Styling:** Modern, responsive CSS layouts.

5. Database Schema Design

The relational database schema is structured to ensure high integrity and normalization. The core relationships are modeled below:



5.1 Main Entities

- **Users:** Stores identity details, password hashes, and user roles (Admin or Student).
- **Topics & Lessons:** Form a hierarchical tree where each Topic has multiple chronological Lessons containing educational content.
- **Quizzes, Questions & Options:** Map modular evaluations. Options carry a boolean IsCorrect field which is strictly evaluated on the server side.

- **User Quiz Attempts:** Logs details about every quiz evaluation, storing the calculated score, counts, and completion timestamps.
- **Achievements:** Stores unlock instances representing student milestones (e.g. `first_lesson`, `perfect_score`).

6. Technical Implementation Highlights

6.1 Server-Side Evaluation & Secure Quiz Submission

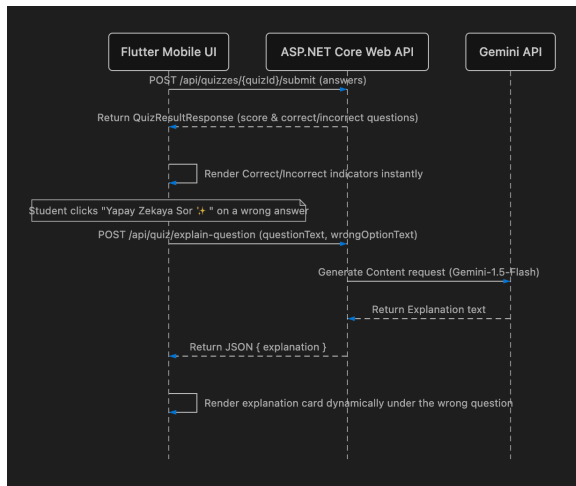
To prevent security leaks (such as intercepting HTTP traffic to inspect correct answers), the backend does not expose `IsCorrect` option flags on the quiz-fetching GET endpoint (`GET /api/topics/{topicId}/quiz`). The correctness is computed server-side in `SubmitQuiz`:

```
csharp
[Authorize]
[HttpPost("quizzes/{quizId:guid}/submit")]
public async Task<IActionResult>
SubmitQuiz(Guid quizId,
SubmitQuizRequestDto request)
{
    // 1. Identify User and Load Quiz
    // 2. Compute Score and Correct
    Answers Count
    // 3. Save UserQuizAttempt to
    PostgreSQL DB
    // 4. Evaluate and trigger
    Achievements dynamically (first_quiz,
    perfect_score, etc.)
    // 5. Return JSON payload detailing
    which answers were correct and which
    option IDs were correct
}
```

6.2 Instant Results Feedback & AI-Tutor Integration

The application implements an **AI-Assisted Explanations Engine** using Google Gemini. Students get immediate feedback on their results screen. For every incorrect response, a custom "Yapay Zekaya Sor ✨" (Ask AI) button is dynamically rendered.

When triggered, it calls `POST /api/quiz/explain-question` providing the question text and the student's selected wrong choice. The backend calls the Gemini API under a custom system instruction, producing a 2-sentence motivational explanation outlining why that specific choice is incorrect in a friendly, conversational tone.



7. Development Milestones & Verification

Milestone	Target Layer	Status	Verification Tool
Database Relational Migrations	Database (Postgres)	Completed	Entity Framework CLI
Server-side submit endpoint	Backend (.NET API)	Completed	dotnet build
AI explain-question endpoint	Backend (.NET API)	Completed	dotnet build & API calls
HTTP client integration wrapper	Mobile (Flutter)	Completed	flutter analyze
Responsive Results UI with Indicators	Mobile (Flutter)	Completed	flutter analyze
Ask AI Button & Inline explanations	Mobile (Flutter)	Completed	Emulator Testing

- **Backend Build Status:** Built successfully with 0 Warnings and 0 Errors using .NET 10.0 SDK.
- **Mobile Analysis Status:** Evaluated with flutter analyze returning No issues found!.

8. Future Work

- **Docker Containerization:** Dockerizing the ASP.NET Core API and PostgreSQL instances for instant cross-platform deployment.
- **CI/CD Pipeline Setup:** Automated GitHub actions running dotnet test and building APK bundles on release branches.
- **Analytics Dashboard:** Building charts in the React Admin panel to visualize average quiz scores, progress tracking patterns, and most-failed questions across the user base.

9. Conclusion

The **LearningApp** platform demonstrates a robust implementation of modern multi-platform software engineering. By decoupling administrative content creation (React), server-side business logic and security validation (ASP.NET Core), and client-side visualization (Flutter), the system ensures high scalability, absolute safety against answer leakage, and an engaging AI-powered tutoring flow.